MEMORIA DE LA PRÁCTICA 2

Alumno: Alberto Joya

1. Obligatorio

1.1. Validación del país

Archivo: datos.js · config.html

```
const paises = ["españa", "portugal", "alemania", "francia", "italia"];

function checkCountry() {
    const country = document.getElementById("countryInput").value.trim().toLowerCase();
    const message = document.getElementById("countryMessage");

    if (country === "" || !paises.includes(country)) {
        message.value = "País no válido";
        return false;
    }

    message.value = "País válido";
    return true;
}
```

Resumen

Comprueba que el país no esté vacío y pertenezca a la lista configurada: España, Portugal, Alemania, Francia o Italia. Se permiten diferencias de mayúsculas y espacios al principio o al final.

Código

paises es el array de valores admitidos. getElementById("countryInput") obtiene el campo y value recupera su contenido. trim() elimina los espacios de los extremos y toLowerCase() normaliza las mayúsculas.

includes(country) comprueba la pertenencia al array. Si el campo está vacío o no coincide, se escribe «País no válido» y se devuelve false. En caso contrario se escribe «País válido» y se devuelve true.

El evento oninput="checkCountry()" del campo ejecuta esta comprobación al escribir. La función también se llama al enviar el formulario para decidir si se puede guardar.

1.2. Longitud de la contraseña

Archivo: datos.js · config.html

```
function checkPass() {
    const password = document.getElementById("password").value;
    const message = document.getElementById("passwordMessage");

    if (password.length < 9) {
        message.value = "Corta: necesita 9 caracteres";
    } else if (password.length < 13) {
        message.value = "Evaluación de longitud: media";
    } else {
        message.value = "Evaluación de longitud: larga";
    }
}
```

Resumen

Muestra una evaluación orientativa de la longitud mientras se escribe. Menos de 9 caracteres se considera insuficiente; de 9 a 12 se muestra longitud media y, a partir de 13, longitud larga. Este indicador no garantiza la seguridad de la contraseña.

Explicación del código
value contiene la contraseña y length obtiene el número de caracteres. La estructura if / else if / else elige uno de los tres mensajes y lo coloca en passwordMessage.

El atributo oninput="checkPass()" conecta el campo con la función. La comprobación que impide guardar una contraseña de menos de 9 caracteres se realiza también en validarFormulario(), explicada a continuación.

1.3. Temperatura y envío del formulario

Archivo: datos.js · config.html

```
function validarFormulario() {
    const temperatureInput = document.getElementById("currentTemperature");
    const temperature = Number(temperatureInput.value);

    if (!checkCountry()) {
        showModal("El país está vacío o no aparece en la lista.");
        return false;
    }

    if (temperatureInput.value === "" || !Number.isFinite(temperature)) {
        showModal("La temperatura actual debe ser un número válido.");
        return false;
    }

    if (temperature < -273.15) {
        showModal("La temperatura no puede ser menor que -273,15 °C.");
        return false;
    }

    if (document.getElementById("password").value.length < 9) {
        showModal("La contraseña debe tener al menos 9 caracteres.");
        return false;
    }

    guardarConfiguracion();

    if (temperature < 22 || temperature > 26) {
        showModal("Aviso: la temperatura está fuera del intervalo recomendado de 22 a 26 °C.");
    } else {
        showModal("Datos correctos. La lectura se puede registrar.");
    }

    return false;
}
```

Reesumen
Valida el país, la temperatura actual y la longitud mínima de la contraseña antes de guardar. Una temperatura fuera de 22 a 26 °C genera una advertencia después del guardado. El código también rechaza temperaturas inferiores a −273,15 °C.

Explicación del código
Number() convierte el contenido de currentTemperature a número. Se comprueba por separado si el campo está vacío y se utiliza Number.isFinite() para descartar valores que no sean números finitos.

Las primeras condiciones llaman a showModal() y devuelven false cuando detectan un dato no válido. Ese retorno termina inmediatamente la ejecución, por lo que no se llega a guardarConfiguracion().

Una vez superadas las validaciones se llama a guardarConfiguracion(), cuya implementación se explica en Extras. Después se comprueba temperature < 22 || temperature > 26: los límites 22 y 26 están incluidos en el intervalo recomendado.

Como el guardado ocurre antes del aviso de rango recomendado, este aviso no impide almacenar la lectura. El formulario usa onsubmit="return validarFormulario()"; el return false final evita el envío HTML y la recarga, ya que la operación se resuelve en el navegador.

1.4. Ventana modal única

Archivo: config.html · datos.js

```
<div id="modal" class="modal" hidden>
            <div class="modal-contenido" role="dialog" aria-modal="true" aria-
labelledby="modalTitle">
                <h2 id="modalTitle">Aviso</h2>
                <p id="modalMessage"></p>
                <button type="button" onclick="closeModal()">Cerrar</button>
            </div>
        </div>


function showModal(message) {
    document.getElementById("modalMessage").textContent = message;
    document.getElementById("modal").hidden = false;
}

function closeModal() {
    document.getElementById("modal").hidden = true;
}
```

Resumen

Reutiliza una misma ventana para mostrar los errores y avisos de validación del formulario, sin alert() ni librerías externas.

Explicación del código

El contenedor modal empieza oculto mediante hidden. En su interior, modalMessage recibe el mensaje y el botón Cerrar ejecuta closeModal(). role="dialog" y aria-modal="true" identifican el contenido como diálogo; aria-labelledby lo relaciona con su título.

showModal(message) asigna el texto con textContent y cambia hidden a false. closeModal() vuelve a poner hidden en true. Se reutiliza el mismo elemento cambiando solamente el mensaje.

En css/iroom.css, .modal ocupa la pantalla con position: fixed e inset: 0; el fondo semitransparente destaca el diálogo. La regla .modal[hidden] usa display: none para ocultarlo.

1.5. Generación del enlace a Google Maps

Archivo: datos.js · config.html

```
function crearEnlaceMapa() {
    const texto = document.getElementById("gps").value.trim();
    const resultado = convertirCoordenadas(texto);
    const mensaje = document.getElementById("mapMessage");

    if (resultado === null) {
        mensaje.textContent = "Coordenadas no válidas";
        return;
    }

    const enlace = "https://maps.google.com/maps?q=" + resultado.latitud + "," + resultado.longitud;
    mensaje.innerHTML = '<a href="' + enlace + '" target="_blank">Ver ubicación en Google Maps</a>';
}
```

Resumen

El botón «Crear enlace al mapa» genera un enlace a la ubicación introducida en el campo GPS. Admite coordenadas decimales y coordenadas en grados, minutos y segundos mediante la función de conversión del siguiente apartado.

Explicación del código

Se lee gps y se eliminan sus espacios exteriores con trim(). convertirCoordenadas(texto) devuelve un objeto con latitud y longitud, o null cuando no reconoce una pareja válida.

Si el resultado es null, se muestra «Coordenadas no válidas» y return termina la función. En caso contrario se construye la URL con el orden latitud,longitud y se inserta un enlace en mapMessage. target="_blank" permite abrir el mapa en otra pestaña.

1.6. Conversión de las coordenadas

Archivo: datos.js

```
function convertirCoordenadas(texto) {
    const partes = texto.split(",");

    // Formato decimal: 40.513, -3.3486
    if (partes.length === 2) {
        const latitud = Number(partes[0]);
        const longitud = Number(partes[1]);

        if (Number.isFinite(latitud) && Number.isFinite(longitud) &&
            latitud >= -90 && latitud <= 90 &&
            longitud >= -180 && longitud <= 180) {
            return { latitud, longitud };
        }
    }

    // Formato DMS: N 40° 30' 45", O 3° 20' 54"
    const patron =
/([NSEOW])\s*(\d+(?:\.\d+)?)\s*[°°]\s*(\d+(?:\.\d+)?)\s*['']\s*(\d+(?:\.\d+)?)\s*[""]/gi;
    const datos = [...texto.matchAll(patron)];

    if (datos.length !== 2) {
        return null;
    }

    let latitud = null;
    let longitud = null;

    for (const dato of datos) {
        const direccion = dato[1].toUpperCase();
        const grados = Number(dato[2]);
        const minutos = Number(dato[3]);
        const segundos = Number(dato[4]);

        if (minutos >= 60 || segundos >= 60) {
            return null;
        }

        let valor = grados + minutos / 60 + segundos / 3600;

        if (direccion === "N" || direccion === "S") {
            if (latitud !== null || grados > 90) return null;
            if (direccion === "S") valor = -valor;
            latitud = valor;
        } else {
            if (longitud !== null || grados > 180) return null;
            if (direccion === "O" || direccion === "W") valor = -valor;
            longitud = valor;
        }
    }

    if (latitud === null || longitud === null) {
        return null;
    }

    return { latitud, longitud };
}
```

Resumen

Transforma las coordenadas introducidas en una pareja decimal. Las direcciones sur y oeste se convierten a valores negativos. Por ejemplo, N 40º 30' 45", O 3º 20' 54" produce aproximadamente 40.5125, −3.348333.

Explicación del código
En primer lugar, split(",") intenta separar dos componentes decimales. Number() los convierte y Number.isFinite() verifica que sean finitos; después se comprueban los límites de −90 a 90 para latitud y de −180 a 180 para longitud.

Si no se acepta el formato decimal, la expresión regular busca componentes DMS. [NSEOW] captura la orientación; los grupos numéricos capturan grados, minutos y segundos. Se admiten diferentes símbolos de grado y comillas. Las banderas g e i permiten buscar todas las coincidencias sin distinguir mayúsculas.

matchAll() obtiene las coincidencias y el operador [...] las convierte en un array. Se exigen dos componentes y se recorre cada uno con for...of. Los minutos y segundos deben ser menores de 60.

La fórmula grados + minutos / 60 + segundos / 3600 produce el valor decimal. N y S se asignan a latitud; E, O y W, a longitud. S, O y W cambian el signo. Las comprobaciones adicionales evitan repetir el mismo eje y limitan el valor de los grados.

Finalmente se devuelve { latitud, longitud }. Si falta uno de los ejes o se detecta una condición no admitida, se devuelve null para que crearEnlaceMapa() muestre el aviso.

1.7. Correo electrónico sin expresiones regulares

Archivo: minombre.html · datos.js

```
<input type="text" id="email">
<button type="button" onclick="validarCorreoSinRegex()">Comprobar sin regex</button>
<button type="button" onclick="validarCorreoConRegex()">Comprobar con regex</button>
<p id="emailMessage"></p>

function validarCorreoSinRegex() {
    const correo = document.getElementById("email").value.trim();
    const partes = correo.split("@");
    let valido = partes.length === 2;

    if (valido) {
        const usuario = partes[0];
        const dominio = partes[1];
        valido = usuario !== "" && dominio !== "" && !correo.includes(" ") &&
            dominio.includes(".") && !dominio.startsWith(".") && !dominio.endsWith(".");
    }

    mostrarResultadoCorreo(valido);
}
```

Resumen
Añade el campo de correo en minombre.html y un botón para comprobar su formato utilizando operaciones sobre cadenas.

Explicación del código
El primer botón llama a validarCorreoSinRegex(). La función obtiene el texto, aplica trim() y lo divide por @. Exigir partes.length === 2 comprueba que exista una única arroba.

Se comprueba que usuario y dominio no estén vacíos y que no haya espacios normales en el texto. includes(".") exige un punto en el dominio; startsWith() y endsWith() impiden que el dominio empiece o termine por punto.

El resultado booleano se entrega a mostrarResultadoCorreo(). La función no utiliza expresiones regulares.

1.8. Correo electrónico con expresión regular

Archivo: datos.js

```
function validarCorreoConRegex() {
    const correo = document.getElementById("email").value.trim();
    const valido = /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(correo);
    mostrarResultadoCorreo(valido);
}

function mostrarResultadoCorreo(valido) {
    document.getElementById("emailMessage").textContent = valido ? "Correo válido" : "Correo no
válido";
}
```

Resumen

El segundo botón comprueba el correo con una expresión regular y muestra el resultado en el mismo lugar que la comprobación por cadenas.

Explicación del código

Los símbolos ^ y $ delimitan el principio y el final del texto. [^\s@]+ exige uno o más caracteres que no sean espacios en blanco ni arrobas. El símbolo @ separa usuario y dominio; \. representa un punto literal.

test(correo) devuelve true si la cadena cumple el patrón y false si no lo cumple. mostrarResultadoCorreo(valido) utiliza el operador ternario para elegir «Correo válido» o «Correo no válido» y lo escribe con textContent.

Las dos comprobaciones son reglas simplificadas de formato. La siguiente tabla recoge el conjunto de ejemplos del enunciado y el resultado de ambas funciones para esos casos.

1.9. Calculadora: operaciones y unidades

Archivo: index.html · calculadora.html

```
<a href="calculadora.html">Calculadora</a>
<button type="button" onclick="calcular('sumar')">Sumar</button>
<button type="button" onclick="calcular('restar')">Restar</button>
<button type="button" onclick="calcular('multiplicar')">Multiplicar</button>
<button type="button" onclick="calcular('dividir')">Dividir</button>
<p>Resultado: <output id="resultado"></output> <span id="unidadResultado"></span></p>

let valorResultado;
let unidadResultado;
function calcular(operacion) {
    const numero1 = Number(document.getElementById("numero1").value);
    const numero2 = Number(document.getElementById("numero2").value);

    if (operacion === "sumar") {
        valorResultado = numero1 + numero2;
    } else if (operacion === "restar") {
        valorResultado = numero1 - numero2;
    } else if (operacion === "multiplicar") {
        valorResultado = numero1 * numero2;
    } else {
        if (numero2 === 0) {
            document.getElementById("resultado").value = "No se puede dividir entre 0";
            document.getElementById("unidadResultado").textContent = "";
            valorResultado = undefined;
            return;
        }
        valorResultado = numero1 / numero2;
    }

    unidadResultado = document.getElementById("unidad").value;

    actualizarResultado();
    guardarEnHistorial(numero1, numero2, operacion, valorResultado);
}

function actualizarResultado() {
    document.getElementById("resultado").value = valorResultado.toFixed(2);
    document.getElementById("unidadResultado").textContent = obtenerSimbolo(unidadResultado);
}

function obtenerSimbolo(unidad) {
    if (unidad === "C") {
        return "°C";
    }
    if (unidad === "K") {
        return "K";
    }
    return "°F";
}
```

Resumen

La página principal enlaza a otro documento HTML con una calculadora. Permite sumar, restar, multiplicar y dividir mediante botones, y muestra el resultado acompañado de Celsius, Kelvin o Fahrenheit según la unidad elegida.

Explicación del código

Los botones pasan a calcular(operacion) una cadena que identifica la operación. Number() convierte los valores de numero1 y numero2. La estructura condicional aplica +, −, * o / y guarda el valor en valorResultado.

Las variables valorResultado y unidadResultado se declaran fuera de las funciones para conservar el estado entre una operación y una conversión. El selector unidad ofrece los valores C, K y F.

actualizarResultado() utiliza toFixed(2) para mostrar dos decimales. obtenerSimbolo() traduce C a °C, K a K y F a °F; el símbolo se escribe junto al resultado.

El control de división entre cero y la llamada a guardarEnHistorial() forman parte del mismo flujo. Se explican por separado en Extras para distinguirlos de las operaciones mínimas.

1.10. Conversión entre escalas

Archivo: calculadora.html

Código

```
function convertir() {
    if (valorResultado === undefined) {
        return;
    }

    const nuevaUnidad = document.getElementById("unidadDestino").value;

    let celsius;
    if (unidadResultado === "C") {
        celsius = valorResultado;
    } else if (unidadResultado === "K") {
        celsius = valorResultado - 273.15;
    } else {
        celsius = (valorResultado - 32) * 5 / 9;
    }

    if (nuevaUnidad === "C") {
        valorResultado = celsius;
    } else if (nuevaUnidad === "K") {
        valorResultado = celsius + 273.15;
    } else {
        valorResultado = celsius * 9 / 5 + 32;
    }

    unidadResultado = nuevaUnidad;
    actualizarResultado();
}
```

Resumen

Convierte el resultado actual entre Celsius, Kelvin y Fahrenheit y actualiza tanto el valor mostrado como su unidad.

Explicación del código

Si todavía no existe un resultado, valorResultado es undefined y la función termina con return. El selector unidadDestino determina la nueva escala.

La conversión se realiza en dos pasos. Primero se pasa el resultado a Celsius: Kelvin resta 273,15 y Fahrenheit aplica (F − 32) × 5 / 9. Si ya estaba en Celsius, se conserva su valor.

Después se pasa de Celsius al destino: Kelvin suma 273,15 y Fahrenheit aplica C × 9 / 5 + 32. Finalmente se actualiza unidadResultado y se llama a actualizarResultado(). Así, las conversiones sucesivas utilizan la unidad real del resultado actual.

Ejemplos: 0 °C equivale a 273,15 K y a 32 °F. La conversión modifica el resultado de la calculadora; no modifica los dos operandos de entrada.

2. Extras

Estas funcionalidades complementan los apartados obligatorios de P2. La personalización del nombre y el cálculo de diferencias se documentan también como funciones complementarias del proyecto continuado de prácticas anteriores.

2.1. Guardado y recuperación de la configuración

Archivo: datos.js

```
function guardarConfiguracion() {
    const configuracion = {
        pais: document.getElementById("countryInput").value,
        ciudad: document.getElementById("city").value,
        gps: document.getElementById("gps").value,
        mapa: document.getElementById("mapUrl").value,
        temperatura: document.getElementById("currentTemperature").value,
        maxima: document.getElementById("maxima").value,
        minima: document.getElementById("minima").value
    };

    localStorage.setItem("configuracion", JSON.stringify(configuracion));
}

function cargarConfiguracion() {
    const guardada = localStorage.getItem("configuracion");
    if (!guardada || !document.getElementById("countryInput")) {
        return;
    }

    const configuracion = JSON.parse(guardada);
    document.getElementById("countryInput").value = configuracion.pais || "";
    document.getElementById("city").value = configuracion.ciudad || "";
    document.getElementById("gps").value = configuracion.gps || "";
    document.getElementById("mapUrl").value = configuracion.mapa || "";
    document.getElementById("currentTemperature").value = configuracion.temperatura || "";
    document.getElementById("maxima").value = configuracion.maxima || "";
    document.getElementById("minima").value = configuracion.minima || "";
}

document.addEventListener("DOMContentLoaded", cargarConfiguracion);
```

Resumen

Conserva país, ciudad, coordenadas, URL del mapa, temperatura actual y límites de alerta para recuperarlos al volver a abrir la configuración. La contraseña y la fotografía no se guardan en este objeto.

Explicación del código

guardarConfiguracion() reúne los valores en un objeto JavaScript. JSON.stringify() lo convierte en texto y localStorage.setItem() lo almacena bajo la clave configuracion.

cargarConfiguracion() obtiene el texto con getItem(). Si no hay datos o la página no contiene countryInput, termina sin intentar rellenar campos. Esta comprobación permite cargar datos.js también desde minombre.html.

JSON.parse() reconstruye el objeto. Cada propiedad se copia al campo correspondiente; || "" proporciona una cadena vacía si falta el dato. DOMContentLoaded ejecuta la recuperación una vez construido el documento.

El almacenamiento pertenece al navegador y al origen de la web; no constituye un registro en un servidor.

2.2. Modo oscuro persistente

Archivo: tema.js · css/iroom.css

```
const botonTema = document.getElementById("botonTema");

function cambiarTema() {
    document.body.classList.toggle("tema-oscuro");

    const oscuro = document.body.classList.contains("tema-oscuro");
    try {
        localStorage.setItem("modoOscuro", oscuro ? "si" : "no");
    } catch (error) {
    }
    botonTema.textContent = oscuro ? "Modo claro" : "Modo oscuro";
}

let modoGuardado = "no";
try {
    modoGuardado = localStorage.getItem("modoOscuro") || "no";
} catch (error) {
    modoGuardado = "no";
}

if (modoGuardado === "si") {
    document.body.classList.add("tema-oscuro");
    botonTema.textContent = "Modo claro";
}

botonTema.onclick = cambiarTema;
```

Resumen

Permite alternar el aspecto claro y oscuro desde las páginas y conserva la elección al navegar o recargar.

Explicación del código

botonTema referencia el botón común. cambiarTema() usa classList.toggle("tema-oscuro") para añadir o quitar la clase del body. Las reglas CSS asociadas cambian los colores de fondo, texto, campos y contenedores.

classList.contains() comprueba si el modo oscuro está activo. Se guarda «si» o «no» en localStorage con la clave modoOscuro. El operador ternario adapta el texto del botón a la siguiente acción disponible.

Al cargar el script se recupera la preferencia. Si su valor es «si», classList.add() activa la clase y el botón muestra «Modo claro». Los bloques try/catch permiten continuar si el navegador no admite el acceso al almacenamiento.

La asignación botonTema.onclick = cambiarTema conecta el botón con la función sin ejecutarla en ese momento.

2.3. Historial de operaciones

Archivo: calculadora.html

```
function guardarEnHistorial(numero1, numero2, operacion, resultado) {
    const nombres = {
        sumar: "+",
        restar: "−",
        multiplicar: "×",
        dividir: "÷"
    };

    const linea = document.createElement("li");
    linea.textContent = numero1 + " " + nombres[operacion] + " " + numero2 + " = " +
resultado.toFixed(2);
    document.getElementById("historial").appendChild(linea);
}

function borrarHistorial() {
    document.getElementById("historial").innerHTML = "";
}
```

Resumen
Añade una línea al historial cada vez que se completa una operación y permite vaciar la lista con un botón. El historial se conserva mientras permanece abierta esa página; no se almacena en localStorage.

Explicación del código
El objeto nombres relaciona cada operación con su símbolo. document.createElement("li") crea una nueva entrada, y textContent combina los operandos, el símbolo y el resultado con dos decimales.

appendChild(linea) incorpora la entrada a la lista historial. calcular() llama a esta función después de actualizar el resultado. Las conversiones de unidad no añaden entradas al historial.

borrarHistorial() asigna una cadena vacía a innerHTML para eliminar todas las entradas. El botón Borrar historial llama a esta función mediante onclick.

2.4. Control de división entre cero

Archivo: calculadora.html · fragmento de calcular()

```
if (numero2 === 0) {
    document.getElementById("resultado").value = "No se puede dividir entre 0";
    document.getElementById("unidadResultado").textContent = "";
    valorResultado = undefined;
    return;
}
```

Resumen
Muestra un mensaje específico cuando se intenta dividir entre cero y evita dejar un resultado disponible para convertir.

Explicación del código
La condición numero2 === 0 se evalúa en la rama de división antes de realizar la operación. Se escribe el aviso en resultado y se borra el símbolo de unidad.

Asignar undefined invalida el resultado anterior. return termina calcular() antes de la división, la actualización y el registro en el historial. convertir() también comprueba undefined, por lo que no convierte ese estado de error.